
Lessons from Sim-to-Real on a 15-DOF Bipedal Robot: Hardware Realities, Reward Brittleness, and the Limits of BC Diffusion for Locomotion

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 We present a reflective account of working with the Open Duck Mini V2, a low-cost
2 15-DOF bipedal robot. The work spanned hardware assembly, PPO policy training
3 in MuJoCo MJX, sim-to-real transfer attempts, and an application of Diffusion
4 Policy via behavior cloning from PPO rollouts. The central outcomes are negative:
5 a PPO policy that dominates in simulation (beating the author-provided baseline
6 on 11/14 metrics) fails to transfer to hardware, and a BC diffusion policy fails
7 catastrophically even in simulation (98% fall rate). Rather than presenting these
8 as successes, we analyze where each stage broke down. We identify three root
9 causes: (1) hardware assembly errors that created unmodeled dynamics, (2) reward
10 overfitting in simulation without real-world validation, and (3) the inadequacy of
11 offline BC-from-distillation for unstable bipedal locomotion without corrective
12 feedback. We conclude with concrete recommendations—system identification,
13 real-world data collection, and DPPO—that would address each failure mode.

14 1 Introduction

15 Sim-to-real transfer remains one of the central challenges in robot learning. The promise is compelling:
16 train policies endlessly in simulation at zero marginal cost, then deploy on hardware. In practice,
17 the gap between simulated and real dynamics—the *sim-to-real gap*—makes this transfer fragile,
18 especially for inherently unstable systems like bipedal robots.

19 This paper recounts a hands-on project with the Open Duck Mini V2, a small, low-cost (~\$200)
20 bipedal robot with 15 degrees of freedom. The work progressed through four stages:

- 21 1. **Hardware assembly** of the physical robot from a kit
- 22 2. **PPO policy training** in MuJoCo MJX, iterated over 28 experiments
- 23 3. **Sim-to-real transfer** attempts with the trained policies
- 24 4. **Diffusion Policy** via behavior cloning from PPO rollouts

25 Each stage produced a negative result or an instructive failure. The trained PPO policy beat the
26 original author’s baseline on 11 of 14 simulation metrics but performed poorly on hardware. The BC
27 diffusion policy achieved a 98% fall rate in simulation. Rather than presenting selective successes,
28 we document these failures honestly and analyze their root causes. We believe this kind of failure
29 analysis is underrepresented in the literature and is often more instructive than success stories.

30 Our contributions are:

- 31 • A detailed analysis of how hardware assembly errors (missing Loctite, mis-initialized joints)
32 create sim-to-real gaps that domain randomization cannot close
- 33 • A case study in reward overfitting: a policy that dominates in simulation but fails on hardware

- An empirical demonstration that BC-from-distillation fails catastrophically on bipedal locomotion, with analysis of why—and how this differs from the manipulation setting where Diffusion Policy succeeds
- Concrete recommendations for future work: system identification, real2sim2real loops, and DPPO

2 Related Work

Sim-to-real transfer. Domain randomization [?] is the standard approach for bridging the sim-to-real gap, treating modeling errors as noise to be averaged over. System identification [?] takes the complementary approach of directly measuring and matching real-world dynamics. Our experience suggests that for bipedal robots with significant hardware variability, SysID is essential and randomization alone is insufficient.

Diffusion Policy. Diffusion Policy [?] applies denoising diffusion models to visuomotor policy learning, demonstrating strong performance on manipulation tasks. DPPO [?] extends this by using a diffusion policy as the policy class within a PPO objective, combining diffusion’s expressive action representation with RL’s corrective feedback loop.

Behavior cloning limitations. The compounding error problem in BC was formalized by [?]. We discuss this in detail in Section 6.3 in the context of our empirical results.

3 The Platform

The Open Duck Mini V2 is a 15-DOF bipedal robot: 10 leg joints (hip yaw/roll/pitch, knee, ankle per leg) and 5 head/neck joints. It runs on a Raspberry Pi 5 with Feetech STS3215 servos and an onboard IMU. The upstream repository provides a complete PPO training pipeline using Brax in MuJoCo MJX, with automatic ONNX export and a runtime for the Pi.

The robot’s appeal is its low cost and open-source toolchain. The challenge is that it is a biped—inherently less stable than a quadruped, and far more sensitive to sim-to-real gaps in joint dynamics, mass distribution, and sensor calibration.

The observation space is 101-dimensional: gyroscope (3), accelerometer (3), velocity command (7), joint angles minus defaults (14), joint velocities (14), three steps of action history (3×14), motor targets (14), foot contacts (2), and gait phase (2). The action space is 14-dimensional: position offsets applied as $\text{motor_targets} = \text{default} + \text{action} \times 0.25$. The policy runs at 50 Hz (20 ms control period) with physics at 500 Hz.

4 Hardware Lessons

4.1 Missing Loctite on Actuator Screws

The assembly guide did not emphasize that threadlocker (Loctite) is essential for all screws on the actuator joints. After several sessions of testing, screws on multiple actuators loosened progressively, introducing mechanical play—backlash and friction that the simulated model did not account for.

The Feetech servos produce enough torque to vibrate screws loose over hundreds of gait cycles. The resulting backlash is asymmetric, velocity-dependent, and differs from joint to joint. The backlash simulation in MuJoCo (`scene_flat_terrain_backlash.xml`) models generic gear play but cannot capture this specific pattern of screw-loosening-induced play.

Lesson. Always use threadlocker on any joint that transmits force. This is a hardware problem that demands a hardware fix, not a software one.

Table 1: PPO policy comparison: custom policy (28 experiments of reward tuning) vs. the original author’s best policy. The custom policy beats the baseline on 11/14 metrics in simulation.

Metric	Custom Policy	Author Baseline	Delta
Stand tracking (angular)	0.907	0.883	+2.7%
Forward tracking (angular)	0.655	0.520	+26%
Backward tracking (linear)	0.229	0.073	+214%
Turn right (angular)	0.526	0.286	+84%
Turn left (angular)	0.547	0.322	+70%
Metrics improved	11 / 14		

75 4.2 Incorrect Neck Joint Initialization

76 The assembly guide did not provide clear instructions for initializing the neck joints (neck_pitch,
77 head_pitch, head_yaw, head_roll) to their zero positions. We suspect the joints were initialized at an
78 offset from where the simulated model expects them.

79 For a biped, this matters more than it might seem. The head assembly has non-trivial mass, and its
80 position affects the center of mass of the entire robot. On a quadruped, a small COM offset is a
81 nuisance. On a biped, it can be the difference between balancing and falling.

82 **Lesson.** Before any sim-to-real deployment, verify every joint’s zero position against the
83 simulated model’s home keyframe. The runtime’s `init_pos` array must exactly match
84 `keyframe("home").ctrl` from the MJCF model.

85 5 PPO Policy Training

86 5.1 Training Pipeline

87 We trained PPO policies using the MuJoCo MJX pipeline on rented cloud GPUs (Vast.ai, RTX
88 4060 Ti). Over 28 experiments, we iterated on reward weights through stacked fine-tuning: starting
89 from a baseline policy, progressively adjusting reward scales and continuing training from the best
90 checkpoint.

91 The key parameter discovery was that `tracking_ang_vel` dominates forward walking quality.
92 Increasing it from the default 6.0 to 14.0 (through a chain of $6 \rightarrow 8 \rightarrow 10 \rightarrow 12 \rightarrow 14$) steadily
93 improved angular velocity tracking and forward gait, at the cost of lateral tracking. The best policy
94 (experiment 28, 159M steps) achieved the results shown in Table 1.

95 5.2 The Sim-to-Real Gap

96 Despite dominating in simulation, the custom policy performed poorly on the real robot. The robot
97 could stand and hold position, but walking was unstable—it would fall after a few steps, especially
98 during turns and lateral movement.

99 In retrospect, this is a classic case of *reward overfitting in simulation*. By aggressively optimizing
100 `tracking_ang_vel` through 28 experiments, we produced a policy that exploits the simulated
101 dynamics to achieve high reward scores. The policy learned behaviors that are brittle: they work in
102 the narrow band of dynamics the simulator provides but fail when the real dynamics diverge.

103 The original author’s policy, while scoring lower in simulation, was validated on real hardware during
104 development. It learned a more conservative gait that is robust to the sim-to-real gap. Our policy
105 learned a more aggressive gait that scores higher in sim but lacks this robustness.

106 **Lesson.** Beating an author-provided baseline in simulation does not mean you have a better policy
107 for the real robot. If your goal is sim-to-real transfer, evaluate on hardware early and often. Simulation
108 metrics are necessary but not sufficient.

Table 2: Sim evaluation: PPO baseline vs. BC diffusion policy. The diffusion policy achieves 98% fall rate and survives only 9 steps on average.

Metric	PPO Baseline	Diffusion BC	Delta
Avg survival steps	100 / 1000	9 / 1000	−91%
Fall rate	0%	98%	+98 pp
Stand tracking (linear)	0.809	0.125	−85%
Action smoothness (rate)	0.053	0.357	6.7× noisier

109 5.3 Domain Randomization Was Not Enough

110 Our primary strategy for sim-to-real transfer was domain randomization during training: varying
111 floor friction, joint PID gains, mass distribution, initial poses, and applying random pushes.

112 The problem is that domain randomization treats the sim-to-real gap as *zero-mean noise* around the
113 simulated parameters. It assumes the real robot’s dynamics are somewhere within the randomization
114 envelope. But when the real robot has systematic biases—loosened screws creating asymmetric
115 backlash, a shifted center of mass, servo responses that differ from the identified model—no amount
116 of randomization centered on the wrong parameters will help. You cannot randomize your way out of
117 a systematic modeling error.

118 6 Diffusion Policy via Behavior Cloning

119 6.1 Motivation and Setup

120 We applied Diffusion Policy [?] to this platform via behavior cloning from PPO rollouts: generate
121 (o_t, a_t) pairs by rolling out the PPO policy in simulation, then train a Conditional U-Net to predict
122 action chunks via denoising.

- 123 • **Dataset:** 4,755 trajectories from the original author’s PPO policy, collected at 50 Hz with
124 randomized velocity commands, friction, initial poses, and impulse perturbations. Total:
125 865,780 steps.
- 126 • **Model:** ConditionalUnet1D (16.4M parameters) with DDIM sampling (16 denoising steps).
127 Action chunk size 16, observation horizon 2.
- 128 • **Training:** 100 epochs, batch size 256, AdamW with cosine LR. Final validation loss:
129 0.0221.

130 6.2 Results

131 The diffusion policy failed catastrophically. Table 2 shows the head-to-head comparison.

132 Only 20% of episodes survived even the stand command. All motion commands caused immediate
133 falls.

134 6.3 Analysis: Why BC Diffusion Fails on Bipedal Locomotion

135 **Compounding error (distribution shift).** BC trains a mapping from observations to actions using
136 expert data. At test time, any small prediction error causes the state to leave the training distribution.
137 The next observation is now out-of-distribution, producing a worse action, compounding until the
138 robot falls. This is the DAGger problem [?]: BC policies are correct only on states visited by the
139 expert, but must act on states produced by their own imperfect predictions.

140 **Action smoothness.** The diffusion policy’s action rate was 5–50× higher than PPO’s. Even with
141 DDIM’s deterministic sampling, the denoised action chunks had high inter-step variance. For a
142 bipedal robot that requires smooth, coordinated joint trajectories, jerky actions are immediately
143 destabilizing.

144 **Unimodal teacher.** Distilling from a single PPO policy produces a unimodal action distribution.
145 The diffusion model only sees one way to respond to each situation and cannot learn the diverse

146 recovery behaviors that PPO implicitly uses to stay upright. PPO developed these through the RL
147 survival reward; the diffusion policy never experienced this pressure.

148 **Locomotion vs. manipulation.** This failure highlights a fundamental distinction between loco-
149 motion and manipulation. Diffusion Policy was demonstrated on manipulation tasks where small
150 action errors are recoverable—a slightly misaligned gripper can be corrected on the next step. In
151 bipedal locomotion, the state space is far less forgiving: a small error in foot placement can cause
152 an unrecoverable fall within one or two steps. This is precisely why DPPO [?], which combines
153 diffusion’s expressive action representation with RL’s corrective feedback, is the natural next step for
154 locomotion.

155 7 What We Would Do Differently

156 7.1 System Identification Before Training

157 The single most impactful change would be system identification (SysID) on the real robot before
158 training. Instead of broad domain randomization centered on generic parameters, we would:

- 159 1. Measure each joint’s actual response to commanded positions at various speeds and loads
- 160 2. Identify the actual backlash profile per joint (likely asymmetric and joint-specific after screw
161 loosening)
- 162 3. Measure the real robot’s mass and COM by physical weighing
- 163 4. Update the MuJoCo model’s joint parameters, friction coefficients, and body masses to
164 match

165 Domain randomization is a workaround for an inaccurate model; SysID fixes the model.

166 7.2 Real-World Data Collection and Fine-Tuning

167 With more time, we would set up a data collection pipeline on the real robot:

- 168 1. Run the author’s best policy on the real robot while recording observations and actions at
169 50 Hz
- 170 2. Use this real-world data to fine-tune via offline RL or to identify systematic observation-
171 action mismatches
- 172 3. Iterate: collect data → update model → retrain → evaluate → repeat

173 This is the “real2sim2real” loop that has become standard in modern robot learning. Even a small
174 amount of real-world data is more valuable for closing the sim-to-real gap than any amount of
175 simulated data.

176 7.3 DPPO Instead of BC Diffusion

177 For the diffusion policy work, the correct approach would be DPPO [?], which uses a diffusion
178 model within an on-policy RL objective. This addresses both the compounding error problem
179 (RL provides corrective feedback) and the unimodal teacher limitation (the policy explores diverse
180 recovery behaviors). BC-from-distillation is a reasonable starting point for manipulation where errors
181 are non-catastrophic, but it is the wrong tool for bipedal locomotion.

182 8 Conclusion

183 This project produced two negative results: a PPO policy that excels in simulation but fails on
184 hardware, and a BC diffusion policy that fails even in simulation. Both failures are informative.

185 The sim-to-real gap was dominated by hardware issues (loose screws, mis-initialized joints) that
186 domain randomization cannot fix. The correct response to a systematic sim-to-real gap is not more
187 randomization but better modeling—system identification to bring the simulated dynamics closer to
188 reality.

189 The diffusion policy failure is a textbook illustration of behavior cloning’s limitations on unstable
190 systems. It reinforces the principle that locomotion requires closed-loop corrective feedback, whether

191 from RL (PPO, DPPO) or from real-world data. BC is well-suited for tasks where small errors are
192 recoverable; bipedal walking is not such a task.

193 The most valuable outcome of this project is not any single result but the understanding of *where*
194 each method breaks down and *why*. For future work, the path forward is clear: SysID to close
195 the sim-to-real gap, real2sim2real loops for iterative refinement, and DPPO to combine diffusion’s
196 expressive action representation with RL’s corrective feedback for bipedal locomotion.

197 **9 Outlook**

198 This project concluded earlier than intended due to time constraints. However, the lessons learned
199 here directly inform ongoing and planned work. The immediate next step is a collaborative project on
200 hierarchical multi-agent policies for a team of four Open Duck robots, where the sim-to-real insights
201 and policy-class comparisons from this warm-up project will be applied in a multi-agent setting.
202 Beyond this, the core takeaways—that system identification matters more than domain randomization,
203 that closed-loop feedback is essential for unstable systems, and that honest failure analysis accelerates
204 learning—transfer across embodiments to upcoming work on quadrupedal locomotion and bimanual
205 manipulation.